RankeDB

Florian Metzger-Noel
2026-04-15 — draft — CC-BY-4.0

Abstract

TODO: introduction, picking up basic idea from previous paper.. bridge to implementation..

The system is delivered as two components: **RankeDB Server**, a self-contained server (Docker Compose stack) exposing the complete data model through a REST API, and **RankeDB Explorer**, a bundled visual interface for navigating the Provenance DAG and the Semantic Graph. Together they constitute a complete, deployable provenance database with an integrated visualization tool.

TODO: here we can tell, that it's a series of papers and which part in the series this one plays..

Note on structure. The paper is organized in two parts. **Part I — The Intuition** (§1) argues why a provenance-first foundation must exist, from the archival tradition through the CS priority to the machine-reading/writing rupture. **Part II — The Foundation** (§2–§5) describes what was built on that intuition: three levels, one graph, one API, and the properties that follow from them. Part II closes by pointing forward to follow-up papers that will continue that investigation by presenting the **first generation of application** on the foundation — the test of whether the philosophy-derived architecture bears load. The paper is a falsifiable bet: if the assumptions hold, later generations of workers and applications will continue to build on the same base; if they do not, the foundation was misjudged.

Part I — The Intuition

Part I builds the case for why a provenance-first foundation must exist. The argument runs in three movements: a 180-year archival tradition that already understood knowledge as inseparable from its chain of attribution (§1.1); a computer-science priority that was identified but never operationalized as a knowledge-graph substrate (§1.2); and an acute rupture in the machine-learning era that makes the old oversight untenable (§1.3). The three converge on a single conclusion (§1.4): the foundation described in Part II is the minimum response to what has been lost.

1. The Problem: Knowledge Without Provenance

1.1. The Archival Tradition

RankeDB (pronounced *run-keh-dee-bee*) is named after Leopold von Ranke (1795–1886), the historian who transformed his discipline by insisting that every historical claim must trace back to a critically examined primary source. Ranke's famous phrase — history *"wie es eigentlich gewesen"*, "as it actually was" — has since been rightly criticized for assuming unmediated access to past reality. RankeDB takes that criticism as foundational: the primary data point is never *"how it was"* but the artifact of a communicative act that reports, claims, or interprets it — an email, a chat message, a voicemail, a document. What RankeDB stores is always someone's utterance about the world, never the world itself. What survives from Ranke's method, intact, is the discipline of attribution: nothing is asserted without its derivation, and nothing is derived without its sources.

1.2. The CS Priority That Was Never Operationalized

Existing systems that address this tension do so partially. Temporal knowledge graphs (Graphiti/Zep, Rasmussen 2025) preserve *when* facts were valid but perform destructive entity summary updates, losing derivation history. Versioned knowledge bases (TerminusDB, Mendel-Gleason et al.) track *what* changed across snapshots but not *why* or *how* knowledge was derived. Immutable databases (Datomic, Hickey 2012 Fluree) preserve all historical states but lack a semantic knowledge layer and do not model derivation chains. No existing system treats the full chain of provenance — from raw source artifact through extraction, normalization, and synthesis — as first-class, queryable knowledge.

1.3. The Rupture: Machines Reading and Writing at Scale

Knowledge management systems face a fundamental tension: they must serve both *current* truth and *historical* understanding. Traditional knowledge graphs optimize for the former — they store what is believed to be true now, updated in place as understanding changes. This design made sense in an era of expensive storage and limited query capacity. It makes less sense in a regime where the ability to present a model with the full derivation history of a belief — including contradictions, revisions, and competing interpretations — may support qualitatively better reasoning than presenting a single consolidated snapshot.

For a rich treatment of what *provenance* has meant across 180 years — from the archival principle of *respect des fonds* through the Semantic Web to the LLM era — we refer the reader to Talisman's essay (Talisman 2026). In this paper we use the term in a narrower, operational sense: the complete derivation chain of a piece of knowledge — the raw source artifact, every intermediate processing step, every tool and configuration involved, and every transformation applied. This is compatible with W3C PROV-DM's Entity/Activity/Agent vocabulary (Moreau & Missier 2013) but makes a stronger commitment: in RankeDB, provenance is not metadata about knowledge — it *is* knowledge, stored in the same graph, queryable through the same API, subject to the same invariants. Each node in the graph is both a statement and the record of how that statement came to be. There is no separate "provenance layer" — the derivation chain is the knowledge, and the knowledge is the derivation chain.

1.4. Convergence: A Foundation, Not a Feature

RankeDB addresses this gap through a structural inversion: the provenance DAG is the system, and everything else — including the semantic knowledge graph — is a view derived from it. It is deliberately **under-prescribed** in how it should be used: the data model preserves every level of detail in parallel — from the raw source artifact up to the semantic triplet — networked by provenance, and leaves the strategy of retrieval and reasoning to the consumer. The follow-up papers will be the first generation of application on that foundation; Part II describes what they will be built against.

Part II — The Foundation

Part II describes what was built from the intuition of Part I: a single graph with three levels, one API, and a small set of invariants that encode the provenance-first commitment. The core properties are presented first (§2), from which the architecture follows as their synthesis (§3), then the reference implementation that enforces it (§4), and finally the worker model through which the graph is populated (§5). Part II closes with a brief forward pointer to the follow-up papers as the first generation of application on this foundation.

2. Core Properties

The properties below presume a few architectural commitments that §3 details in full. At a glance: RankeDB is a single graph organized into three levels — **Sources** (Level 0, ingested external artifacts), **Cognition** (Level 1, derivations produced by workers), and **Semantics** (Level 2, projected entities and

relations). Every node carries provenance edges back through its inputs all the way to the sources from which it was derived. The sections that follow use this vocabulary without further comment.

2.1. Everything Is Knowledge

RankeDB makes no distinction between data, metadata, and provenance. Every claim made *about* the graph is itself a node in the graph, with its own provenance:

- a classification ("this node belongs to the finance domain"),
- a summary ("this is a condensed version of the conversation at node X"),
- an alias ("this node refers to the same person as node Y"),
- a worker log ("this node was created by a worker of type X with configuration Y").

This principle — *everything is knowledge* — eliminates the need for separate metadata systems, tagging taxonomies, or logging of worker activity as additional infrastructure: all of these are expressible as nodes in the graph, derived from the same sources, subject to the same provenance and immutability guarantees, and queryable through a single API.

From this ontological flatness follows a structural claim. If every claim is a node with provenance, then what is the *primary* content of the system? In conventional knowledge graphs, claims are primary and provenance is an annotation layer bolted on top — an afterthought that explains where things came from. RankeDB rejects this split.

Provenance is not an annotation on the knowledge — it *is* the knowledge. Every derivation, every thought, every projected fact is itself a node in the graph, linked to the inputs it was derived from. There is no "real" layer above and a "provenance" layer below; it is one graph, and the knowledge and its derivation are stored together.

This inversion has concrete operational consequences: operations that would require complex graph surgery in a conventional system become simple view operations in RankeDB. Reprocessing sources with better tools produces new nodes alongside old ones — no migration required. Filtering out results from an obsolete worker is a query parameter, not a data operation. Evaluating competing interpretations of the same source is a traversal, not a diff between snapshots.

2.2. Immutability and Accumulation

RankeDB is strictly append-only. No node or edge is ever modified or deleted through runtime operations. When new information contradicts existing knowledge, the contradiction is represented as a new node — not as an update to the old one. Both coexist in the graph, each with full provenance.

This design is a deliberate bet on the trajectory of language model context windows. Systems that destructively consolidate today — merging entity summaries, deduplicating facts, compacting histories — optimize for current retrieval efficiency at the cost of inferential depth. RankeDB optimizes for a future in which a model able to traverse the full derivation history of a belief as needed — contradictions, revisions, and competing interpretations — may produce better reasoning than one given only a consolidated summary.

2.3. Under-Prescription: A Base for Evolution

Reprocessing as the motivation.

Sources are preserved and immutable, so the derivations built from them can be re-run as tools improve. A better OCR engine in 2028 produces better text extractions from a 2024 photograph; a better summarizer in 2030 produces a better summary from a 2026 conversation. Old and new outputs coexist (§2.2); the consumer chooses which to prefer — typically by filtering on the worker that produced them.

The derivations at the cognitive and semantic levels of the graph can be rebuilt from the sources in Level 0 — partially. Deterministic workers (format conversion, normalization, deduplication) reprocess byte-exact. Non-deterministic workers (LLM extraction, summarization, synthesis) produce epistemologically similar but not byte-identical output on re-runs, and the further downstream a derivation sits from a non-deterministic worker, the less it can be reconstructed at all.

Everything is preserved down to the raw material in Level 0 — always available for a fresh attempt.

The design commitment: under-prescription.

Because every level of detail is preserved (§2.1) and every node carries provenance to its inputs, consumers can traverse the graph at whatever granularity they need — down to the raw bytes, up through every derivation. We call this stance **under-prescription**: the database is deliberately short on commitments that would constrain future use. RankeDB does not decide in advance which level of detail is the right one to query, which granularity is best for which question, or which projection best serves which application. The alternative — committing to a specific retrieval strategy or summarization scheme — makes today's consumers faster but freezes the design around today's capabilities: if entity resolution gets better in 2028, a session-centric store from 2026 has already discarded the evidence needed to exploit the improvement.

Under-prescription supports strategy pluralism in both dimensions: **across time**, tomorrow's better strategies run over the same data without migration; **at any given time**, multiple strategies run in parallel — one walking associative connections at L2, another pulling raw spans from L0, a third tracing provenance chains in L1. Agents can compete, cooperate, or coexist; each picks its level without negotiating with the others.

The design principle: **capture well now with as few decisions as possible that lead to future constraints.** §2.1 says *provenance is the content.* §2.2 says *do not throw knowledge away.* §2.3 says *do not commit to a single way of using what you kept.* Together these describe a database that prioritizes headroom over optimization for today: usable now, slower than a system tuned specifically for current agent capabilities, but built to get better as those capabilities evolve — faster processing, larger working volumes, richer reasoning over long chains of derivation.

An honest assessment: building the infrastructure described in this paper is the straightforward part. The hard problem is what runs on top — how workers bootstrap a useful Semantic Graph from raw sources, how entity resolution scales in a personal-knowledge context, how consumers navigate the accumulated history. Those are the subjects of the companion papers, and the test of whether this substrate was worth building. The companion papers will describe *one way* to populate and consume this base using today's tools. Neither will be the final answer; both will be first-generation consumers of a substrate designed to outlive them.

3. Architecture

RankeDB is a single connected graph — the **Knowledge Graph (KG)** — organized into three levels (see Figure 1). The KG has two subgraphs, defined by edge type:

- The **Provenance Graph (PG)** contains all nodes (L0, L1, L2) and all provenance edges (`provenance/input`, `provenance/worker`). PG is a strict DAG — always acyclic, regardless of which levels the edges span. It records how every piece of knowledge came to be.
- The **Semantic Graph (SG)** contains only L2 nodes and relation edges (`relation/head`, `relation/tail`). SG may contain cycles — it is a graph of associations, not of derivation.

Together: KG = PG ∪ SG. Every node appears in PG; L2 nodes additionally appear in SG. Every edge belongs to exactly one of the two subgraphs.

The three levels classify content, not processing order:

- **Level 0 — Sources.** A forest within PG: every source has at most one parent (a format conversion, a normalization, an item extracted from a bulk container), and each originally ingested artifact is a root.
- **Level 1 — Cognition.** Nodes in Level 1 may combine inputs from any level. Together with Level 0, Level 1 populates the bulk of the Provenance Graph.
- **Level 2 — Semantics.** Entity and relation nodes. Each relation is described by a relationship node with zero or more `relation/tail` edges (subject) and `relation/head` edges (object) — supporting ambiguity and structural unknowns. Level 2 is populated by projection workers, not a deterministic function of Level 1 (§3.1.3).

Levels classify content, not processing order: a worker may cite nodes from any level as provenance inputs.

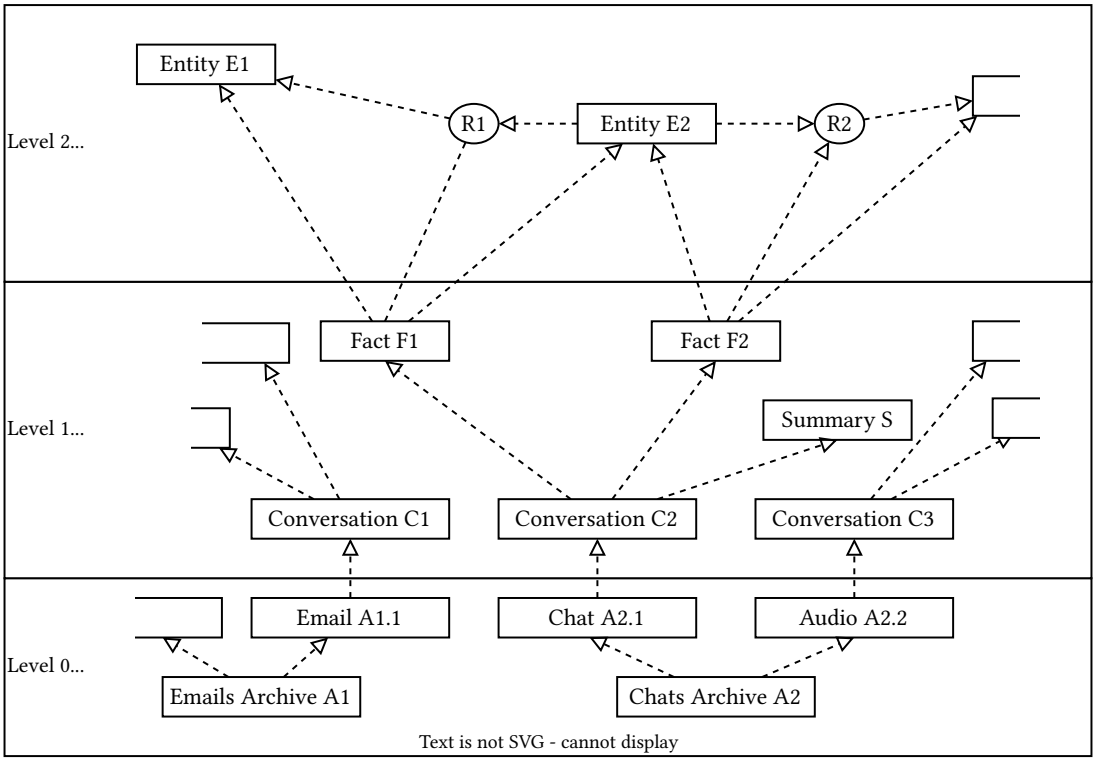

Figure 1: The three storage levels of RankeDB. Concrete node labels such as Email, Conversation, Fact, and Summary are illustrative examples. RankeDB defines content type categories (e.g. `source/conversation`, `classification/entity`) and leaves encodings and application-specific types to the application layer.

The graph is strictly append-only. Once written, a node or edge is never modified or deleted — beliefs later found to be false remain in the graph as knowledge of what was once held true, annotated by new nodes that record their falsification. Removal is possible only as an administrative operation, treated as a fork of the original graph (as in functional programming over immutable data structures). §2.2 develops the consequences.

3.1. Nodes

All nodes in the graph share a common format. Level-specific extensions are introduced in §3.1.1, §3.1.2, and §3.1.3; there is no overlap between level-specific fields.

Content and identity are separated: a node carries its payload in `content` together with `content_sha256` and `content_len` for integrity and size, while `id` is the node's identity in the graph.

For L0 root artifacts, `id` is deterministic from `content_sha256` — this is what makes ingestion idempotent: re-uploading the same bytes maps to the same root node. For all other nodes (derived L0 nodes, L1 derivations, L2 projections), `id` is synthesized independently, because two nodes with identical content but different provenance are distinct knowledge.

Field	Purpose
`id`	Node identity (deterministic from `content_sha256` for L0 root artifacts; synthesized otherwise)
`title`	Short subject line — what this node is about, not what kind of thing it is (optional)
`content`	Payload (text or bytes, interpreted per `encoding`)
`content_sha256`	Cryptographic hash of `content`
`content_len`	Byte length of `content`
`content_type`	Category and type (dispatch key for workers and consumers)
`encoding`	MIME-style `class/format` (e.g. `text/eml`, `image/png`); dispatch key for workers
`created_at`	When the node entered the graph

The `content_type` field follows a two-part pattern: `category/type`. RankeDB defines the categories and a set of foundational types; applications may extend the types within each category.

The `encoding` field follows a MIME-style pattern: `class/format`. The class is hardcoded and small — `text`, `image`, `audio`, `video`, `application` — and doubles as a machine-readable policy hint (only `text/*` is treated as text; everything else is binary). The format is the specific syntax (e.g. `text/eml`, `text/whatsapp`, `image/png`, `application/pdf`) and is the primary dispatch key for reactive workers. Formats are application-extensible; each format is a micro-project: a parser, quickly written, easily tested. Nodes can wait patiently until a parser for their format becomes available.

Date fields that carry real-world time are accompanied by a `_blur` sibling expressing temporal fuzziness as a duration (e.g. `30d` = ±30 days, `0` = precise). Blur defaults to `0`, so it is opt-in: workers that produce precise timestamps leave it alone; workers that produce imprecise dates — "around 2018," "mid-20th century" — set it to the scale of fuzziness. Consumers extend range queries by the blur to cover fuzzy matches; the exact interpretation (hard cutoff, weighted match) is a consumer concern. Blur is orthogonal to `confidence`: blur captures how precise the timing is; `confidence` captures how sure we are of the claim itself. System-captured dates (`created_at`) are precise by construction and have no blur.

3.1.1. Level 0: Sources

Level 0 is the region of the graph that holds ingested external artifacts. An artifact may be a communicative act (an email, a chat transcript, a letter, a voicemail), a document (a book, a contract, an article), a perceptual capture (a photograph, a recording), a machine observation (a sensor reading, a transaction log), or structured data (a spreadsheet, a database export). Every artifact enters the graph as a node, self-describing through attached metadata and its own content. A source node may be an original artifact or a source derived deterministically from another source — e.g. unpacked from a bundle (e.g. an individual conversation extracted from a bulk chat export), converted from another format (e.g. TIFF to PNG), or cleaned and normalized (e.g. stripped of html tags). All of these remain sources: captured artifacts of the world, not knowledge *about* them.

Level 0 is the archive. It does not claim truth about the world — it just stores the artifacts ingested. That's our ground truth, the fixpoint against which every derivation can be traced.

In addition to the common fields, L0 nodes carry:

Field	Purpose
`artifact_created_at`	Original creation date of the external artifact
`artifact_created_at_blur`	Temporal fuzziness around `artifact_created_at`, as a duration (default `0`)
`origin`	Ingest pathway
`original_name`	Original filename

Content types:

Level 0 holds only `source/*` content types. RankeDB defines four source types and one container type. The design principle is *few types, many encodings*: the diversity of the world lives in encodings, not in the type system. Format-specific diversity within a content type (e.g. `text/eml`, `text/whatsapp`, `text/telegram` for conversations) is reconciled by normalization workers that produce new Level 0 nodes with the same content type but a canonical encoding (e.g. `text/plain`). Normalization changes only the format, not the kind of thing — the node is still a source artifact. By the time Level 1 workers pick up a conversation for cognitive processing, source-format diversity is irrelevant — workers will likely process only normalized content (though they have full access to the graph and can process whatever they need).

Content type	What it captures	Examples
`source/conversation`	Communicative act with sender and receiver, even if implicit. An invoice is a conversation (sender ⟶ receiver). An article is a conversation (author ⟶ readers). What *kind* of conversation it is — invoice, contract, smalltalk — is determined by a classification worker in Level 1, not at import.	email, chat, letter, voicemail transcript, article
`source/contact`	Structured representation of a person or organization's contact information, normalized from any source format. Not yet an entity — the mapping from contact to person is resolved by workers in Level 2.	VCF/vCard, phone book entry, address list row, social profile
`source/media`	Audio, visual, or audiovisual capture. Content is opaque until a worker processes it — could be a voicemail, art, a surveillance recording, or a meeting.	photo, video, audio recording, screen capture
`source/record`	Objective, machine-generated observation of world-state. Not human expression — structured readings from sensors, APIs, instruments.	GPS positions, weather readings, stock prices, bank transactions
`source/data`	Structured information that does not fit the above categories. Defined by exclusion: not a communicative act, not a perceptual capture, not a machine observation, not a contact. Application layer decides boundary cases.	spreadsheets, configuration files, database exports

`source/bulk`	Container of other sources. Unpacked by workers into individual source nodes. The bulk node serves deduplication across repeated exports — if a contained source already exists (same hash), it is skipped.	ChatGPT export, WhatsApp backup, Gmail archive, photo library export

Invariants:

- Root artifact identity is deterministic from content. Re-ingesting the same bytes yields the same node.
- Source nodes are self-describing. Metadata is sufficient for full reconstruction (modulo worker nondeterminism).
- Writes are idempotent. A duplicate `PUT` has no effect.

3.1.2. Level 1: Cognition

Level 1 adds derived knowledge to the Provenance DAG. Its nodes are derived from source nodes or other derived nodes through processing by external tools (workers). Every derived node requires at least one input and one tool attribution. The DAG is strictly acyclic because derivations cannot be circular: a node cannot be derived from its own output.

We call that process *cognition* as a metaphor for signal processing in the human brain, spanning low-level operations (edge detection in the visual cortex, phoneme recognition) up to abstract reasoning. The term does not imply consciousness, awareness, or intent — we use it as shorthand for *information processing that extracts knowledge from lower levels of the graph.*

Together with Level 0, Level 1 forms the complete Provenance DAG. Where Level 0 provides the roots — the sources — Level 1 stores the derivation history: not just what is believed, but *how it came to be believed.* This history is itself knowledge: queryable, traversable, and available as context for downstream consumers.

L1 nodes use only the common fields (§3.1); there are no L1-specific node fields. A derivation produced by a 2024 language model and one produced by a 2028 model from the same source are both preserved as competing interpretations, each with a full provenance chain that includes the worker configuration in effect at the time.

Content types:

Every node in Level 1 is the output of a worker interpreting, classifying, extracting, summarizing, or reasoning about the graph. The content type categories distinguish *what kind* of derivation it produces.

Level 1 content types follow the same `category/type` pattern as Level 0. The following categories are part of the RankeDB architecture; the types within each category are application-defined. Examples given are illustrative only — RankeDB does not commit to any particular structure within a type.

Category	Purpose	Examples
`conversation/*`	L1 representations of conversations, beyond raw source format.	`conversation/transaction`, `conversation/interview` (never by source format)
`image/*`	L1 representations of images, beyond raw source format.	`image/photo`, `image/diagram` (never by source format)
`video/*`	L1 representations of videos, beyond raw source format.	`video/meeting`, `video/lecture` (never by source format)

`classification/*`	A worker's statement about a node: what it is, who appears in it, what it concerns. Classification nodes bridge the Provenance DAG and the Semantic Graph — they live in Level 1 with full provenance and their edges project into Level 2.	`classification/entity` (who/what was identified), `classification/content` (what kind of thing is this), `classification/topic` (what is this about)
`observation/*`	A worker's statement about relationships between nodes — grouping, contradiction, correlation, sequence, gaps. The natural output of analytical workers that traverse the graph rather than processing individual nodes.	`observation/contradiction`, `observation/alias`, `observation/grouping`
`summary/*`	Condensed representation of one or more nodes.	by length, audience, or purpose
`fact/*`	Extracted factual claim with provenance to the node that supports it.	by domain, or by confidence threshold

Source types and Level 1 types are not 1:1. A `source/record` containing bank transactions can resolve into `conversation/transaction` nodes — sender, receiver, amount as message. A `source/media` might resolve into `conversation/transaction` or into `image/diagram`. The source type in Level 0 captures how an artifact entered the graph; the Level 1 type captures what it means.

Dependencies among Level 1 types are emergent from the workers and their usage of the infrastructure, not prescribed by the architecture: a `conversation` worker may wait for `classification/entity` results before it can resolve participants, producing a natural ordering without the architecture having to enforce one.

Invariants:

- The graph is acyclic within this level. No node can transitively depend on itself.
- Every node has provenance. No node exists without at least one input edge and one tool attribution.

3.1.3. Level 2: Semantics

Level 2 holds the Semantics of the knowledge graph, structured as a Semantic Graph: a property graph optimized for associative traversal and retrieval. It is a first-class level of the graph, populated by projection workers that read Level 1 and produce the entity and relation nodes that make associative retrieval possible. If Level 2 were merely a deterministic view of Level 1, it would be redundant; it exists as its own level because associative traversal over the full Provenance DAG would be prohibitively expensive at scale, and because the cognitive work of deciding *what* to project is itself a worker activity.

Every node in Level 2 has a provenance edge back into Level 1. Relation nodes carry zero or more `tail` edges (subject) and zero or more `head` edges (object) to the entity nodes they connect — each with its own `confidence`. Semantic connections within Level 2 may be cyclic; the acyclicity of the Provenance DAG applies only to provenance edges (§3.2).

Relation labels are natural-language strings, not formal ontology predicates. The ontology is not predefined — it emerges from the data as workers extract and normalize relations over time.

In addition to the common fields, L2 nodes carry:

Field	**Purpose**

`valid_from`	Start of temporal validity window
`valid_from_blur`	Temporal fuzziness around `valid_from`, as a duration (default `0`)
`valid_until`	End of temporal validity window
`valid_until_blur`	Temporal fuzziness around `valid_until`, as a duration (default `0`)
`confidence`	Confidence score

Content types:

Level 2 defines two foundational categories — entities and relations — each with a fixed architectural shape. Entity subtypes name a minimal upper-ontology that many applications will share; applications may extend with their own subtypes.

Category	Purpose	Foundational subtypes (applications may extend)
`entity/*`	Projected nodes representing identifiable things in the world.	`entity/person`, `entity/organization`, `entity/place`, `entity/thing`, `entity/work`, `entity/idea`, `entity/event`, `entity/role`
`relation/*`	Reified semantic relations between entities, with head/tail edges.	`relation/alias` (two entities refer to the same thing), `relation/part_of` (structural composition), `relation/has_role` (entity holds a role, time-bounded), `relation/family` (familial relationship)

The foundational subtypes are a *shared vocabulary*, not a privileged class. RankeDB does not enforce or validate them at the storage layer — an application that ignores them and defines its own types is architecturally equivalent. What the foundational set provides is a coordination point: consumers walking the graph (entity-resolution libraries, UI renderers, analytical tools) can assume that `entity/person` means a human individual, `relation/has_role` is time-bounded, and so on. Apps adopting this vocabulary become interoperable with tools built against it; apps inventing their own are responsible for interoperability themselves. Like every other L2 relation, aliases carry confidence and compete with other claims — consumers decide how to weight them, not the architecture.

Atomic creation of relations.

A relation node and all its head/tail edges are created as one atomic unit by a single worker from a single L1 fact. The relation node's provenance edge to the L1 fact covers the entire creation — node and edges together. No head or tail edge is ever added to an existing relation node later; new information produces a new relation node (immutability, §2.2).

This convention eliminates the need for per-edge provenance: since edges are never created independently of their relation node, the node's provenance is sufficient.

A relation node has zero or more `tail` edges and zero or more `head` edges, each to an entity node, each carrying its own `confidence`. The reading convention is: **"tail IS relation TOWARDS head"** — the tail is the subject, the head is the object. For example: Alice(tail) –[sister_of]–> Bob(head) reads as "Alice is sister of Bob."

This models four cases naturally:

- **Definite.** "Alice is Bob's sister." One tail (Alice), one head (Bob), both at full confidence.
- **Ambiguity.** "Bob or Charlie is Alice's brother." One tail (Bob at 0.7 / Charlie at 0.3), one head (Alice) — competing candidates within one claim.

- **N-ary.** "Susi and Tina like dancing tango." Two tails (Susi, Tina, both at full confidence), one head (tango) — an inherently multi-party relationship.
- **Structural unknown.** "Someone is Alice's brother, but we don't know who." Zero tails, one head (Alice) — the graph knows what it doesn't know.

Invariants:

- Every L2 node has a provenance edge to Level 1.
- Relation nodes and their head/tail edges are created atomically. Edges are never added to an existing relation node.
- Relation labels are natural-language; no formal ontology is required.

3.2. Edges

Every edge in the graph has a `type`. Four types exist, in two classes:

Provenance edges form the strict DAG — always acyclic, regardless of which levels they span.

- **`provenance/input`** — connects a derived node to an input it was derived from (a source, a fact, a relation, or any other existing node). Every non-root node must have at least one.
- **`provenance/worker`** — connects a derived node to the `worker/config` node that produced it. Exactly one per derivation. This is how the graph records who created what, with which configuration.

Each provenance edge carries a `run_id` property identifying the worker run that produced it. This is what enables administrative operations such as purging a defective run — the edges produced by that run are removed, orphaned nodes follow.

Semantic edges connect reified relation nodes to their head and tail entities. They live in Level 2 only and are permitted to form cycles, because the Semantic Graph is a graph of associations, not a graph of derivation. Each semantic edge carries its own `confidence` and `run_id`.

- **`head`** — connects a relation node to an entity that is the object of the relation.
- **`tail`** — connects a relation node to an entity that is the subject of the relation.

Semantic edges are always created atomically with their relation node (§3.1.3) — provenance on the relation node covers the entire unit.

Three further design decisions follow from treating the graph as a single data structure:

- **Parent relationships are edges.** An L0 node derived from another L0 node (a format conversion, a normalization, an item unpacked from a bulk container) is connected by a provenance edge, not a `parent` field. Denormalization may appear at the storage layer as an optimization; it is not part of the node format.
- **Worker configurations are themselves nodes.** A worker's identity and configuration at a point in time are stored as L1 nodes (content_class `worker`, content_type `config`), which a derivation links to via a `provenance/worker` edge. This gives worker configurations their own provenance and lineage.
- **Every node has provenance.** No node exists without at least one incoming `provenance/input` edge and exactly one `provenance/worker` edge (except L0 root artifacts, which are roots by definition).

4. Reference Implementation

4.1. RankeDB

The API exposes RankeDB as a single graph, enforcing the invariants described in §3. Beneath the API, the graph lives in Postgres — all nodes, all edges, across all three levels. The raw bytes of each node's content live in an S3-compatible object store, keyed by content hash; Postgres stores the hash and, for nodes eligible under a caching policy, a copy of the content inline.

The API is the sole interface. There is no query language in the traditional sense — the API *is* the query language, imperative rather than declarative. Workers, the Explorer, and any future application consume the same interface. Storage-engine choices and caching policies are invisible to API consumers.

The reference deployment runs as a Docker Compose stack on a single host.

4.1.1. Postgres: the graph

Postgres holds the full graph — all node metadata across all three levels and the edges between them. Every ingested source, every derivation, and every projected entity or relation has a row here, keyed by `id`, carrying the fields defined in §3.1 plus its `content_sha256` hash.

Postgres also holds:

- A `content_cached` column with inlined content for nodes that are eligible under a caching policy (for example, `encoding_class = 'text' AND content_len <= :threshold`). The policy is a config knob and can be raised or lowered at runtime; a background worker fills or evicts the cache accordingly. Correctness does not depend on cache state — the API falls back to S3 on miss.
- Full-text search indices via `tsvector` over cached text.
- Vector embeddings via pgvector, in a separate table with a foreign key to the content; embeddings are regenerable at any time with a different model or chunking strategy.

4.1.2. S3: content blob storage

The reference deployment uses any S3-compatible object store — S3 is a de-facto standard whose API surface has been stable since 2006, and migration between providers (Backblaze B2, Cloudflare R2, AWS, MinIO, others) is a single `rclone sync` away. Blobs are stored keyed by their content hash (SHA-256 in this deployment). Buckets are configured with versioning (as a guard against accidental deletion during development) and with Object Lock (WORM) for production, which enforces immutability at the storage layer.

We use the object store as *dark storage*. Two choices combine to create this discipline: we route all access through the API (workers never hold S3 credentials and cannot reach blobs directly), and we deliberately do not list or enumerate bucket contents. The API resolves every blob read from Postgres's `content_sha256` field; a compromised worker cannot enumerate what is stored, and Postgres remains the sole discovery path for what lives in the graph.

4.1.3. The API contract

The API is the only way into RankeDB. It enforces two core properties:

- **Immutability.** Once written, a node is never modified. Corrections are new nodes that reference what they correct.
- **Provenance.** Every derivation has edges to its inputs, including the worker configuration node that produced it — no derivation exists in the graph without them. Worker configurations are stored as graph nodes (`worker/config`) so that worker details themselves gain provenance. The guarantee is naturally scoped to what was given to the system — it cannot attest to what was never recorded.

4.1.4. Forking and backups

Content-addressed blob storage makes forks of the database cheap. Because blobs are addressed by their content hash, two or more graph instances can share a single blob pool without copying bytes — only Postgres needs to be duplicated, and the graph is small relative to the blobs. This enables experimentation, A/B testing of worker pipelines, and isolated development against production data.

A full backup has two parts. Postgres holds the full graph — all node metadata, edges, indices, and cached content. The S3 blob pool holds the raw bytes of every node's content, keyed by content hash. From a Postgres backup together with the blob pool, the system's state is fully recoverable; regrowing cognitive and semantic content from an L0-only backup is subject to the limits of reprocessing (§2.3).

4.2. RankeDB Explorer

RankeDB Explorer is a bundled visual interface for navigating and inspecting the data model through the API. It is the first application built against RankeDB.

The Explorer serves three purposes:

- **Provenance inspection.** Given any node in the Semantic Graph (Level 2), the Explorer traces its full derivation chain through Cognition (Level 1) down to the Sources (Level 0). A user can follow any fact back to the raw source that produced it.
- **Graph exploration.** The Semantic Graph can be navigated visually — entities, relations, temporal validity, confidence scores.
- **Architecture validation.** If the Explorer can render full provenance chains, temporal graphs, and cross-level traversals through the API, so can any downstream application — agent systems, dashboards, export tools.

In the current phase, RankeDB is primarily a research tool, and a database without a way to see its contents is not usable as one.

5. Workers

Workers are external processes that read and write the graph through the API. The concrete design of a worker pipeline will be the subject of the companion paper on RankeDB Workers; this section sketches the categories we anticipate, based on patterns that are natural for the data model but have not yet been built at scale.

We expect two broad categories to emerge in practice. **Reactive workers** would poll for unprocessed nodes whose content type matches their profile and produce new nodes of a different content type — format converters, bulk-archive unpackers, normalizers, fact extractors. **Analytical workers** would traverse the graph more freely, searching for contradictions, gaps, or patterns across existing nodes. Both would interact through the same API; the distinction is in their traversal strategy.

Workers could be LLM-based (entity extraction, summarization, synthesis), deterministic (format conversion, deduplication, normalization), or hybrid. RankeDB is agnostic to the nature of a worker — it records only the provenance of what the worker produces: which inputs were consumed, which worker configuration was in effect (linked via `provenance/worker` edge), and the run identifier carried on the edges the worker created (§3.2).

A single node could be processed by multiple workers, or by successive versions of the same worker. Old and new outputs coexist with full provenance (§2.2); consumers would select between them through query parameters — for example, filtering on the most recent worker run.

6. Related Work

6.1. Temporal Knowledge Graphs: Graphiti/Zep

Graphiti (Zep, 2024–2025) is the closest existing system to RankeDB in the LLM context management space. It builds temporal, provenance-aware knowledge graphs using FalkorDB or Neo4j, with bidirectional episode indices and temporal validity windows. Facts are invalidated rather than deleted.

However, Graphiti performs destructive entity summary updates, has no content-addressable source region comparable to RankeDB's Level 0, and embeds provenance as annotation on the knowledge graph rather than treating it as the content itself. RankeDB can be understood as an extension of Graphiti's philosophy — adding immutability, first-class sources, and the architectural inversion that makes provenance the substrate rather than an annotation.

6.2. Versioned Knowledge Bases: TerminusDB

TerminusDB provides Git-like versioning (branch, merge, time-travel) over an RDF knowledge graph using append-only delta encoding. It captures *what* changed across versions but not *why* — there is no derivation chain, no source archive, and no concept of workers as provenance-tracked processors. Its foundational structure is a versioned graph, not a provenance DAG.

6.3. Immutable Databases: Datomic and Fluree

Datomic (Hickey, 2012) operationalizes Pat Helland's "Immutability Changes Everything" thesis as an append-only database of immutable datoms. Fluree combines an append-only ledger with a semantic graph database. Both capture temporal history but not epistemic history — they record *when* facts changed but not *how knowledge was derived from sources through processing chains.*

6.4. W3C PROV-DM

The W3C PROV Data Model provides a formal vocabulary for provenance (Entity, Activity, Agent, wasGeneratedBy, wasDerivedFrom, used). RankeDB's internal model is semantically compatible with PROV-DM — nodes map to Entities, worker runs to Activities, tools to Agents — but RankeDB does not depend on or implement the W3C stack (RDF, SPARQL, OWL). PROV-DM compatibility exists at the conceptual level, enabling potential export or interoperability without architectural coupling.

6.5. Nanopublications

Nanopublications (Kuhn & Dumontier, 2014) are immutable, content-addressable scholarly assertions with embedded provenance. They share RankeDB's commitment to immutability and provenance-per-assertion but are a flat collection of independent assertions — they do not form a derivation DAG connecting assertions through chains of processing, and they do not support a semantic graph layer.

6.6. TODO: Additional prior art (currently missing from §6, must add)

The following systems and traditions are flagged by PDF1 or PDF2 as meaningfully close to RankeDB but are not currently covered. Each is a new subsection to write.

6.7. The Identified Gap

No existing system combines: (a) a content-addressable immutable source archive, (b) an append-only Provenance DAG as the primary data structure, (c) a semantic knowledge graph as a materialized view with per-edge provenance, and (d) natural-language relations with emergent ontology. Each component has mature prior art; the architectural composition is novel.

7. Discussion

7.1. The Context Window Bet

RankeDB's append-only accumulation model is a bet on a specific technological trajectory: that models, retrieval strategies, and reasoning capabilities will keep improving, and that systems holding rich inferential history will be able to exploit those improvements as they arrive. The bet is not about cramming everything into prompt context; it is about keeping the material available so that larger, richer, better-targeted slices can be asked for as capabilities grow. If the trajectory holds, systems that

destructively consolidate today will be unable to reconstruct the inferential context that RankeDB preserves. If it does not, RankeDB pays a storage cost for history that cannot be effectively utilized.

Current trends support the bet. Context windows have grown from 4K tokens (2022) to 1M+ tokens (2026), with inference speed improving, costs per token declining by orders of magnitude and retrieval strategies maturing alongside. The architectural question is not whether models *can* process full provenance chains, but when they can do so at acceptable latency and cost for interactive use.

The bet is also a deliberate rejection of the three paths the field currently takes to equip chat assistants with long-term memory. Wu et al. (2025) summarize them as follows:

> "To equip chat assistants with long-term memory capabilities, three major techniques are commonly explored. The first approach involves directly adapting LLMs to process extensive history information as long-context inputs (Beltagy et al., 2020; Kitaev et al., 2020; Fu et al., 2024; An et al., 2024). While this method avoids the need for complex architectures, it is inefficient and susceptible to the 'lost-in-the-middle' phenomenon, where the ability to utilize contextual information weakens as the input length grows (Shi et al., 2023; Liu et al., 2024). A second line of research integrates differentiable memory modules into language models, proposing specialized architectural designs and training strategies to enhance memory capabilities (Weston et al., 2014; Wu et al., 2022; Zhong et al., 2022; Wang et al., 2023). Lastly, several studies approach long-term memory from the perspective of context compression…"
>
> — Wu et al., *LongMemEval* (ICLR 2025), arXiv 2410.10813v2

Each of these approaches commits the memory solution *to the language model itself* — by expanding its context, modifying its architecture, or compressing what it receives. **RankeDB commits to none of them.** The memory lives as a structured, provenance-addressable database *outside* the model. The reader decides at query time what slice of the available data to consume; the model sees only the slice chosen for it; the rest remains intact and addressable for a different slice next time, by the same model or a different one. The bet on context window growth is not a bet on cramming everything into the prompt — it is a bet that, when model capabilities improve, consumers will be able to ask for larger, richer slices from the same base without reprocessing their inputs and without changing the database. The memory problem and the modeling problem are decoupled by construction.

7.2. Toward a CRDT-Compatible Architecture

An add-only monotonic DAG is provably a Conflict-Free Replicated Data Type (CRDT) — it can be replicated across distributed nodes and always merged into a consistent state without coordination. This property, while not exploited in the current single-node design, suggests that RankeDB's architecture could natively support decentralized, coordination-free knowledge management. This connection between provenance DAGs and CRDTs appears unexplored in the literature.

7.3. Design Rationale: What the Architecture Makes Possible

Wu et al. (2025, *LongMemEval*) identify five core long-term memory abilities — **information extraction**, **multi-session reasoning**, **knowledge updates**, **temporal reasoning**, and **abstention** — as the coverage axes for long-term memory systems. The companion papers on workers and on chat/memory agents will describe how a pipeline and a consumer stack actually deliver these abilities. This section answers the complementary question: *why does the database look the way it does?*

RankeDB does not guarantee any of the five — achieving them is the work of consumers built on top, and the database's role is to make that work easier by keeping every ability reachable. Each architectural choice in §2 and §3 was made to **refuse to foreclose** on the abilities — reachable by

some consumer strategy — without committing to any particular one. A system that commits to one granularity, one retrieval path, one indexing scheme, or one consolidation policy makes some subset of the abilities cheap and the rest expensive or impossible. RankeDB leaves all of them reachable, and leaves the strategy to the consumer.

- **Multiple levels of detail in parallel (§2.3).** *Keeps information extraction and multi-session reasoning possible from the same data.* Raw dialogue, intermediate derivations, and semantic triplets all exist at once, so a consumer can descend to the exact utterance for specific recall *or* aggregate at the entity level for cross-session synthesis — without one undermining the other.
- **Provenance as substrate (§2.1).** *Keeps abstention and knowledge updates possible.* A consumer can check whether a claim has a provenance chain and refuse to answer if it does not — an architectural precondition for any abstention policy above. And because a superseding node links to what it supersedes, current-vs-historical selection is a queryable property, not something lost in a consolidation pass.
- **Timestamps on every node and edge (§3.1.2, §3.1.3).** *Keeps temporal reasoning possible as a first-class query primitive.* Two temporal dimensions are carried at all times: when the underlying event occurred (from source metadata or explicit in-content mentions) and when the node entered the graph (transaction time). Questions about *when* have direct answers; a consumer does not have to reconstruct temporal order from implicit cues.
- **Temporal validity on L2 edges (§3.1.3).** *Keeps knowledge updates possible without losing history.* Every L2 edge carries `valid_from` and `valid_until`, not just a creation timestamp. A fact true from February to April coexists with a fact true from April onward; both remain retrievable, and which one answers a given query is a view-configuration choice left to the consumer.
- **Append-only over content-addressable source (§2.2, §3.1.1).** *Keeps knowledge updates and abstention possible at the infrastructure level.* Nothing is ever mutated or overwritten, and the raw source is byte-identical to what was ingested. History cannot be corrupted by "updating" a fact, and a claim that cannot be verified today can be re-verified tomorrow against the unchanged source.

The five abilities fall out as consequences of the provenance-as-substrate inversion (§2.1) and the under-prescription principle (§2.3), not as design targets chosen in advance.

7.4. Outlook: What the Substrate Makes Possible

RankeDB is scoped as a personal knowledge system; several capabilities that fall outside that scope are nonetheless expressible within the substrate and worth sketching as directions for future work or later papers.

Access control as a derived property. In a multi-user deployment, visibility could be expressed entirely within the graph rather than bolted on as a system feature. Every input artifact would have an owner — a user group, hierarchically organized. A node would be visible to a user if and only if all of its inputs are visible to that user; at the root (Level 0), this would mean ownership by one of the user's groups. Visibility would propagate through the provenance graph automatically: a node derived from one public and one confidential source is confidential; changing visibility at a source propagates to every derivation. This would be *compliance by architecture, not by policy*, and the classification of nodes into visibility groups would itself be a node with provenance — produced by a worker, subject to revision, queryable like any other knowledge. Designing and implementing this is beyond the scope of a personal-knowledge paper but is a natural extension of the provenance-first substrate.

Further outlook items to be added here as the scope of future papers clarifies.

8. Conclusion

RankeDB proposes a structural inversion in knowledge system design: provenance as substrate rather than metadata, knowledge graph as view rather than source of truth. The architecture — immutable source archive, append-only Provenance DAG, Semantic Graph as materialized view — is a composition of individually well-understood components whose integration has not been previously proposed.

The core thesis is that knowledge graph history is not metadata — it is knowledge. By treating all provenance, all classifications, all assessments as first-class nodes in the same graph, RankeDB eliminates the distinction between data and metadata, between knowledge and knowledge-about-knowledge. Everything is knowledge. Everything is graph.

The system is designed for a technological regime that does not yet fully exist — one in which language models can efficiently consume full derivation histories. This is a deliberate architectural bet, analogous to developing a 3D game engine before consumer hardware can render it in real time. Systems that accumulate inferential history today will be positioned to provide qualitatively richer context to capable models tomorrow. Systems that destructively consolidate will not be able to reconstruct what they have discarded.

References

- Helland, P. (2015). Immutability Changes Everything. CIDR 2015.
- Hickey, R. (2012). Datomic: A Database of Flexible, Time-Based Facts.
- Kuhn, T. & Dumontier, M. (2014). Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data. ESWC 2014.
- Mendel-Gleason, G. et al. TerminusDB: An Open Source Model Driven Graph Database for Knowledge Graph Representation.
- Moreau, L. & Missier, P. (2013). PROV-DM: The PROV Data Model. W3C Recommendation.
- Nelson, T. (1965). A File Structure for the Complex, the Changing, and the Indeterminate. ACM/CSC-ER.
- Rasmussen, P. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory.
- Sikos, L.F. & Seneviratne, O. (2020). Provenance-Aware Knowledge Representation: A Survey. Data Science and Engineering.
- Takan, S. (2023). Knowledge Graph Augmentation: Consistency, Immutability, Reliability, and Context. PeerJ Computer Science.

Draft v0.1 — April 2026